

A DATA-DRIVEN COMPARISON OF HOW JUNIOR AND SENIOR DEVELOPERS USE NATURAL LANGUAGE TO GENERATE REACT COMPONENTS

ATHIRA ALLAVEEDU ASHOK

ST86950

M.Sc. Computer Science: Software Engineering

Supervisor: *Dr. sc. ing., Associate Professor Samuel Moveh.*

Outline

1. Research Objectives
2. Object and Subject of the Research
3. Background and Research Gap
4. Methodology
5. Results
6. Limitations and Future Work
7. Conclusion

Objectives

Goal of this Research

To investigate how developer experience influences the formulation of natural language prompts for generating React components and how these prompts affect the quality of the generated code.

RQ1

In what ways do junior and senior developers differ in their formulation of natural language prompts for generating React components, and do these differences result in measurable differences in the quality of AI-generated React components ?

RQ2

Can clustering techniques identify meaningful patterns in prompting behaviour and code quality variables, and do these clusters correspond to developer experience levels?

RQ3

Which prompt and code characteristics most strongly distinguish the two experience-level groups?

Object and subject of the research

Object of the Research – Analysis of the relationship between prompt quality and AI-generated React code quality among junior and senior developers.

Subject of the Research – Natural language prompts written by junior and senior developers for generating React components using ChatGPT.

Background and Research Gap

The AI-Assisted Coding Landscape

- AI-assisted coding tools like ChatGPT are increasingly changing software development.
- React is one of the most widely used frontend frameworks globally.
- Prompt quality influences the quality of AI-generated code.
- Prompt engineering has emerged as an important skill in AI-assisted software development.

Research Gap

- Most existing studies focus on evaluating AI model performance rather than developer behaviour.
- Limited research compares how junior and senior developers naturally formulate prompts.

Methodology

Participants

150 software developers
(n=150)

Junior (n=75): < 3 years
experience

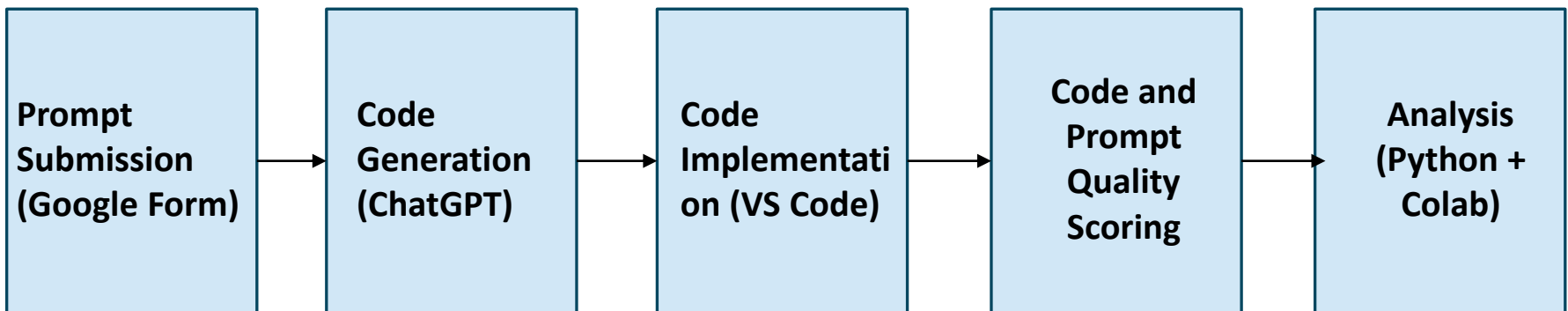
Senior (n=75): $\geq$ 5 years
experience

Recruited via professional
network

Task Design

- One prompt per participant – React Todo Application via Google Form.
- Submitted individually to ChatGPT in isolated sessions by author.
- React components implemented and tested in VS Code.
- Statistical analysis performed using Welch's t-test, Cohen's d, Correlation Analysis and K-means Clustering.

Data Collection and Analysis pipeline



Evidence - Data Collection

150 prompts collected from software developers through Google Forms

Form Responses			
Time	1. What is your c	2. Years o	3. Your Prompt for the React Todo Application :
2/1/2026	Senior / Lead Develop	5+ years	Todo application using React and Vite. Features: Add / remove / toggle tasks Filtering and active count
2/1/2026	Junior	1 - 3 years	I need to create a todo app in react js using vite with create or add, delete and notifier after completing the task...need an optimized code to
2/1/2026	Senior / Lead Develop	5+ years	Design and implement a Todo application using React (Vite). Focus on: Centralized state via useReducer Clear action-driven updates Stateless UI components where possible Tasks should follow a consistent schema (id, text, completed, createdAt). Handle validation at the form level before dispatching actions. Use a custom hook for localStorage persistence. Optimize rendering with memoization and avoid storing derived state. Include filtering logic and a footer summarizing pending tasks.
Junior	1 - 3 years	Core features:	Create a Todo List application using React JS and Vite. The app should allow users to add a new task, delete it, and mark it as cor
Senior / Lead Develop	5+ years	Build a classic Todo List app in React using Vite.	I want a clean, modern single-page todo application with these exact features:
Junior	Less than 1 ye	i need a todo app with normal design a textbox with a blue button with 4 functionalities select,update,delete,add.	
Senior / Lead Develop	5+ years	Create a Todo Application using React.js.	Requirements: - Use functional components and React Hooks (useState).
Senior / Lead Develop	5+ years	Build a React Todo application with Vite.	Add tasks via input field.
Senior / Lead Develop	5+ years	Create a modern, responsive Todo List application using React and Vite. Use functional components and React Hooks (useState).	The app should include the following features: Add Task: An input field and an 'Add' button to create new tasks.
Junior	1 - 3 years	Create a todo list application using react and vite and tailwindcss. The application should have simple and elegant and minimalist	
Junior	Less than 1 ye	Create a Todo app using React with Vite, including basic components for adding, deleting and completing tasks.	

Evidence –Todo Application

Junior To-Do Application Output

Todo App

Enter task...

task 1

Senior To-Do Application Output

To-Do List

Enter a task...

☒ ALL ☐ PENDING ☐ COMPLETED

1 tasks remaining

☐ task5/18/2026, 7:44:09 PM

React Todo applications generated from junior and senior developer prompts and executed in Visual Studio Code.

Evaluation Metrics

Prompt Quality Variables

- Clarity : Ease of understanding requirements.
- Specificity : Level of technical detail provided, such as state management or CSS.
- Structural organisation : Use of headers, bullet points or logical flow.
- Prompt Length (chars): Objective count of characters in submitted prompt.
- Prompt Word Count: Objective count of words in submitted prompt.

Code Quality Metrics

- Cyclomatic Complexity : Measures code logic complexity.
- Maintainability Index : Predicts ease of future updates.
- Readability Score : Measures how easily a human developer can understand the code.
- ESLint Error Count: Static linting (React ESLint default config)
- AI Response Time: Time from prompt input to full source code from ChatGPT.

Tools: ChatGPT, React js, Visual Studio Code, ESLint , Node.js script, Python

Result: Prompt Analysis

Senior Avg. Prompt Length

676.31
characters

Junior Avg. Prompt Length

202.21
characters

Senior Prompt Quality Score

Clarity: 4.99 /5
Specificity: 4.67 /5
Structure: 4.31 /5

Junior Prompt Quality Score

Clarity: 3.52 /5
Specificity: 2.53 /5
Structure: 2.15 /5

Prompt Quality Dimensions (1-5 scale) – Senior Vs Junior

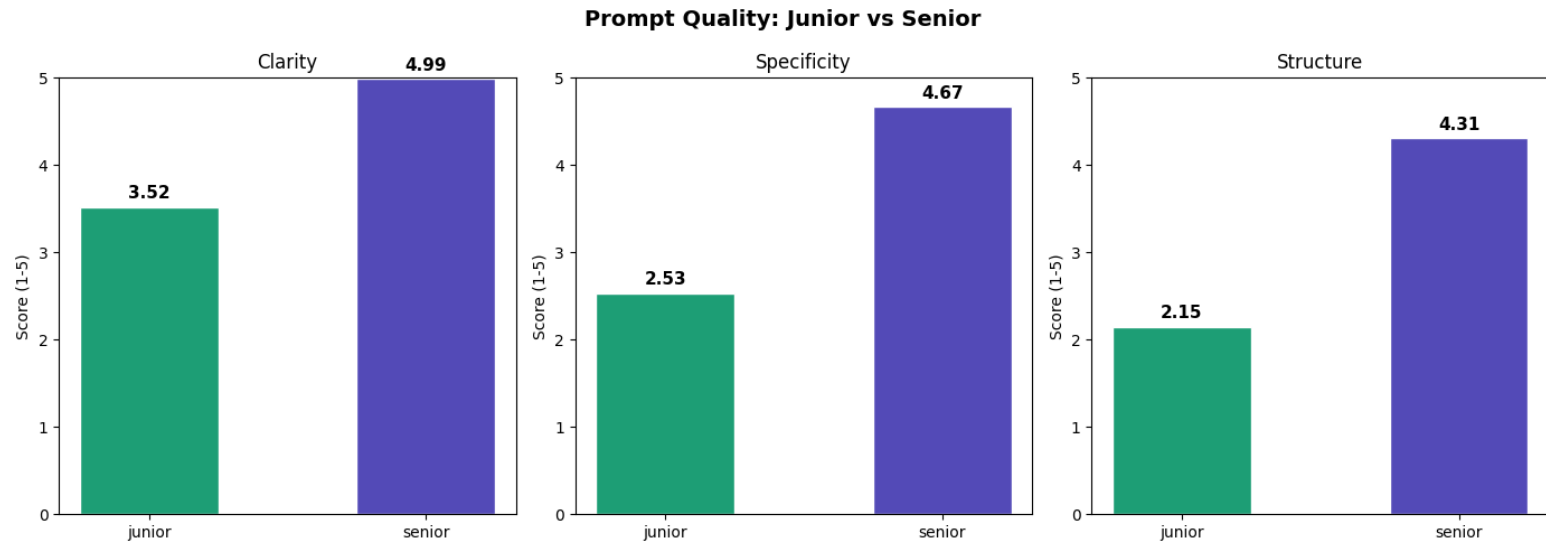


Figure 1. Comparison of Prompt Quality Between Junior and Senior Developers

Result: Code Quality

Metric	Senior	Junior	Verdict
ESLint Errors	5.3% (4/75)	17.3% (13/75)	Senior
Cyclomatic Complexity	3.87 (SD 2.92)	4.70 (SD 1.71)	Senior
Maintainability Index	48.80 (SD 11.06)	37.58 (SD 5.46)	Senior
Readability Score	6.78 (SD 2.12)	7.38 (SD 1.34)	Senior (lower is better)
AI Response Time (sec)	21.32s	13.23s	Longer – reflects prompt length

Table 1 . Comparison of Code Quality Metrics Between Junior and Senior Developers

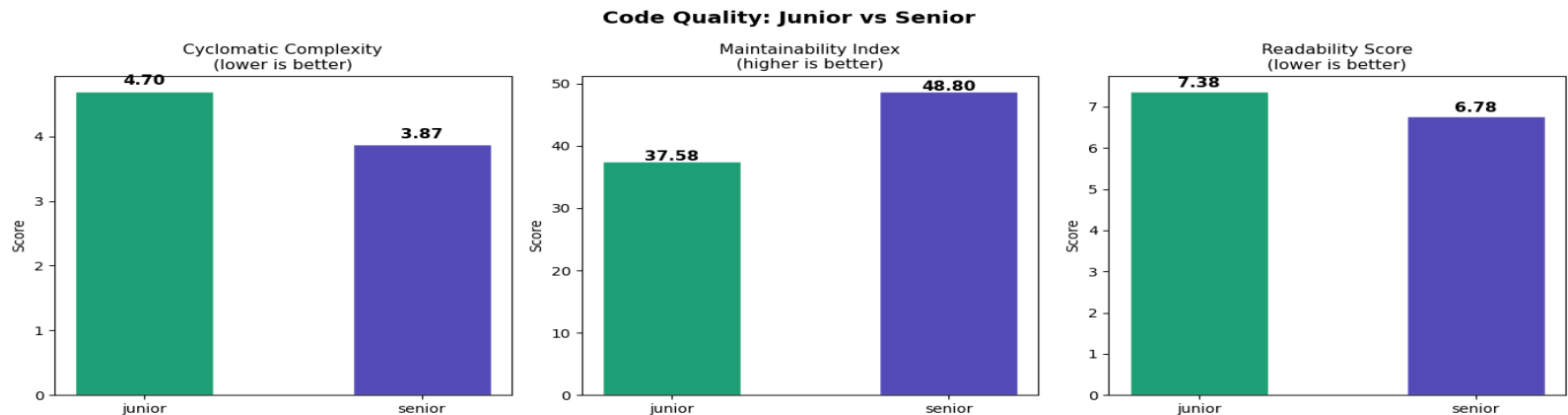


Figure 2. Comparison of Code Quality Metrics

Result: Statistical Significance (Welch's t-test)

Variable	t-statistic	p-value
Prompt Length	-12.137	< 0.001
Word Count	-11.202	< 0.001
Clarity (1-5)	-17.340	< 0.001
Specificity (1-5)	-19.662	< 0.001
Structure (1-5)	-22.910	< 0.001
AI response time	-10.190	< 0.001
ESLint errors	2.345	0.021
Cyclomatic Complexity	2.120	0.036
Maintainability Index	-7.873	< 0.001
Readability score	2.069	0.041

Table 2. Welch's t-test Results for Prompt and Code Quality Metrics

Shapiro-Wilk normality testing showed that all variables were non-normal ($p < 0.001$). Therefore, Welch's t-test was selected because it is robust to non-normality and unequal variances for large sample size ($n = 75$ per group).

Result: Effect Size Analysis (Cohen's d)

Variable	Cohen's d	Magnitude	Threshold
Prompt Length	1.982	Large	$d > 0.8$
Word Count	1.829	Large	$d > 0.8$
Clarity	2.832	Very Large	$d > 2.0$
Specificity	3.211	Very Large	$d > 2.0$
Structure	3.741	Very Large	$d > 2.0$
AI response time	1.664	Large	$d > 0.8$
ESLint errors	0.383	Small	$d > 0.5$
Cyclomatic Complexity	0.346	Small	$d > 0.5$
Maintainability Index	1.286	Large	$d > 0.8$
Readability score	0.338	Small	$d > 0.5$

Table 3. Effect Size Analysis Using Cohen's d

Result: Response Time and Prompt Length

Key Observation

Senior prompts are $\sim 3.35\times$ longer (676 vs 202 chars, $t=-12.137$, $p<0.001$). AI takes 21.32s vs 13.23s ($t=-10.190$, $p<0.001$). Despite longer wait, senior prompts produce significantly better code across all metrics.

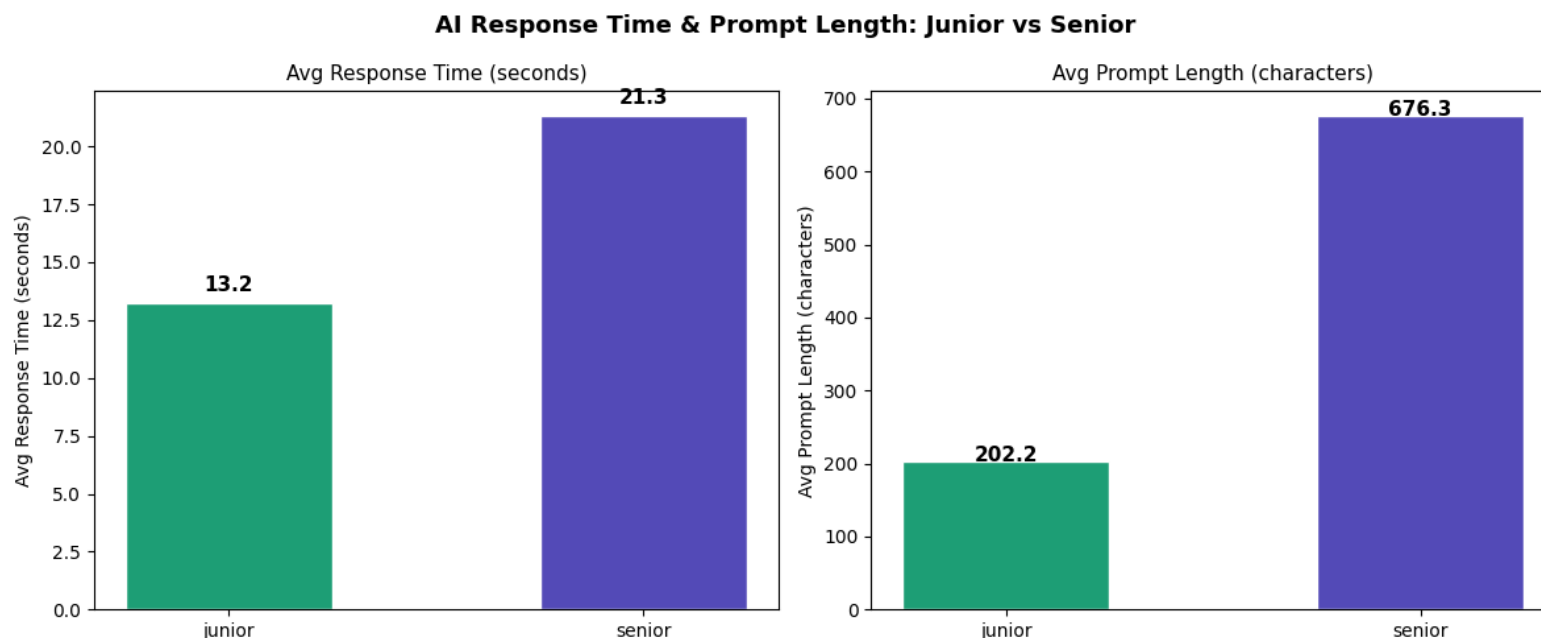


Figure 3. Average Response Time and Average Prompt Length

Correlation Analysis

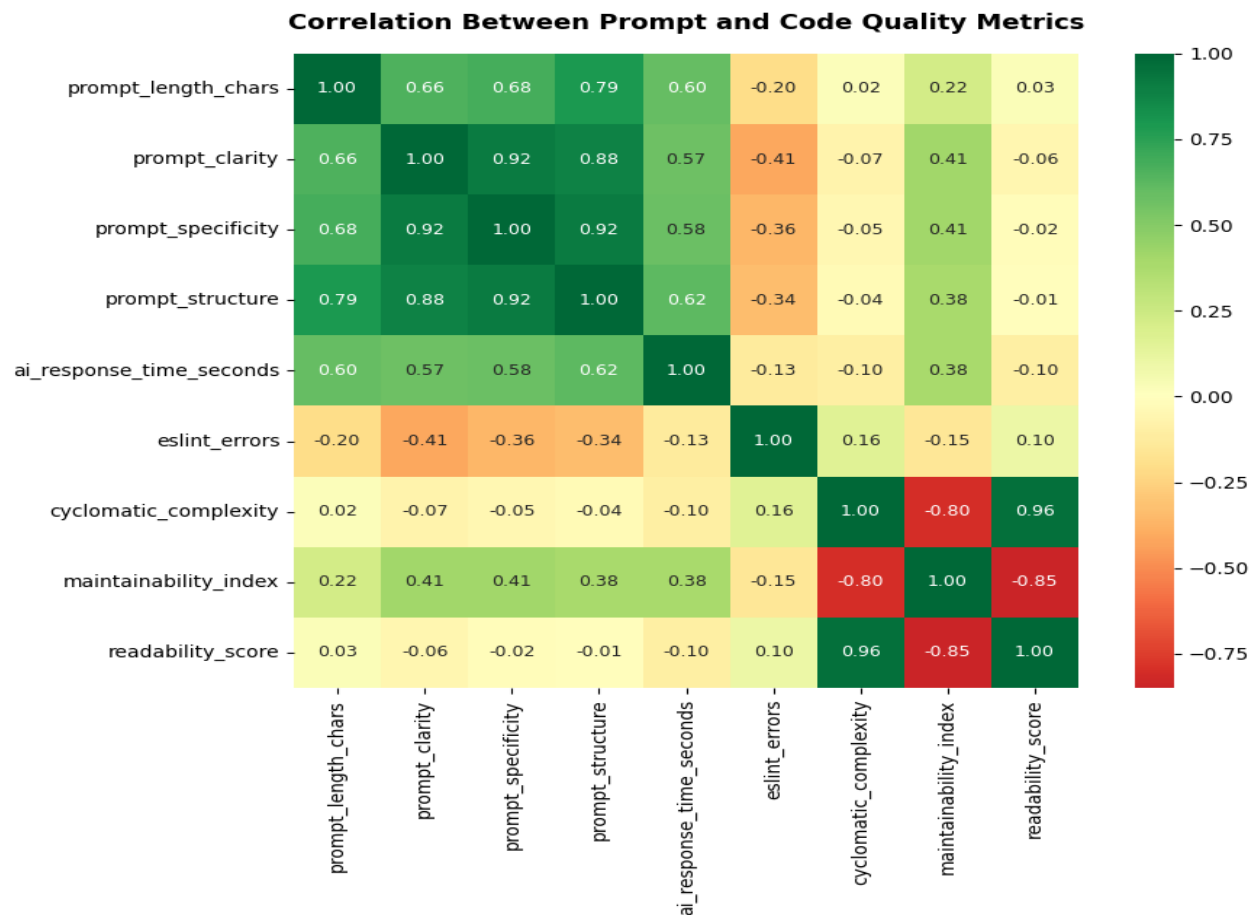


Figure 4. Correlation Analysis Between Prompt and Code Quality Metrics

- Prompt specificity <-> Maintainability Index : positive correlation
- Prompt Structure <-> Maintainability Index : positive correlation
- Prompt length <-> AI response time : positive correlation ($r=0.598$, $p < 0.001$)

K-means Clustering Analysis

Key Finding

Cluster 0 – 82 developers (72 junior + 10 Senior)

- Avg prompt length: 253 chars . Clarity: 3.65
- Maintainability Index : 36.06 . Cyclomatic Complexity:5.43
- Cluster 1 – 68 developers (3 junior + 65 senior)
- Avg prompt length: 662 chars . clarity 4.99
- Maintainability Index: 51.78 . Cyclomatic Complexity:2.90
- Silhouette Score = 0.4781
- Elbow max inertia drop at k = 2

K-Means Clustering Results

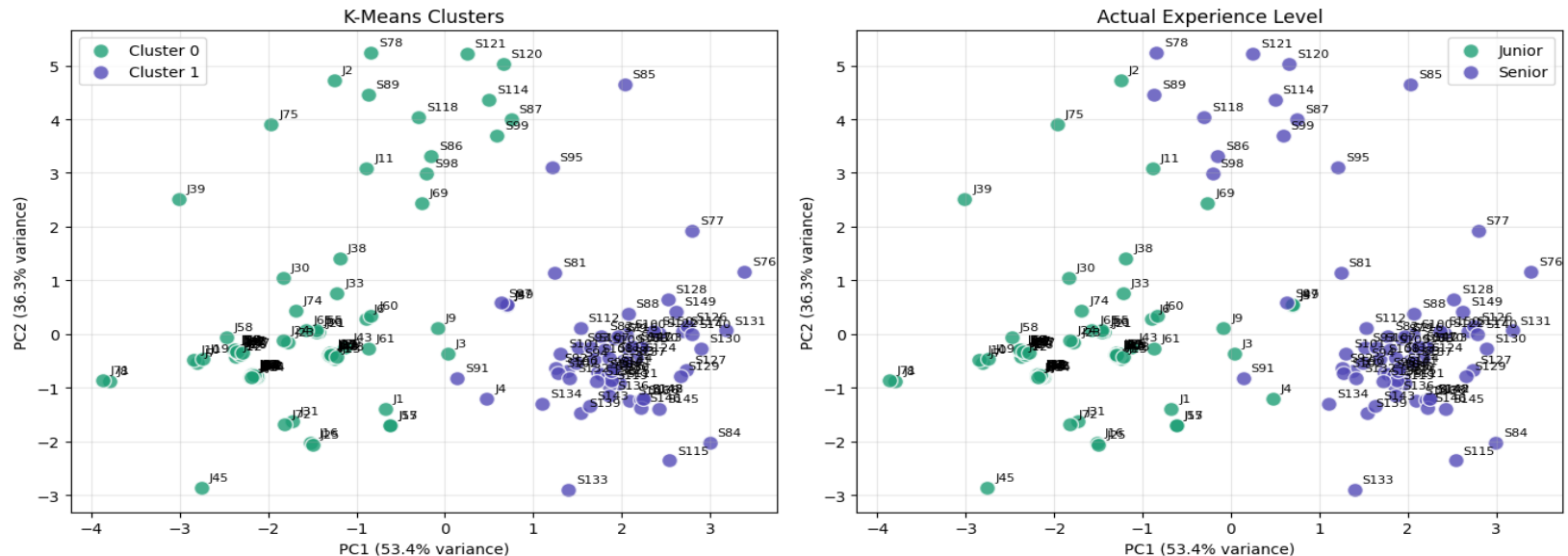


Figure 5. PCA Visualization of K-Means Clusters

Cluster Validation: Elbow Method and Silhouette Score

Elbow Method for Optimal k

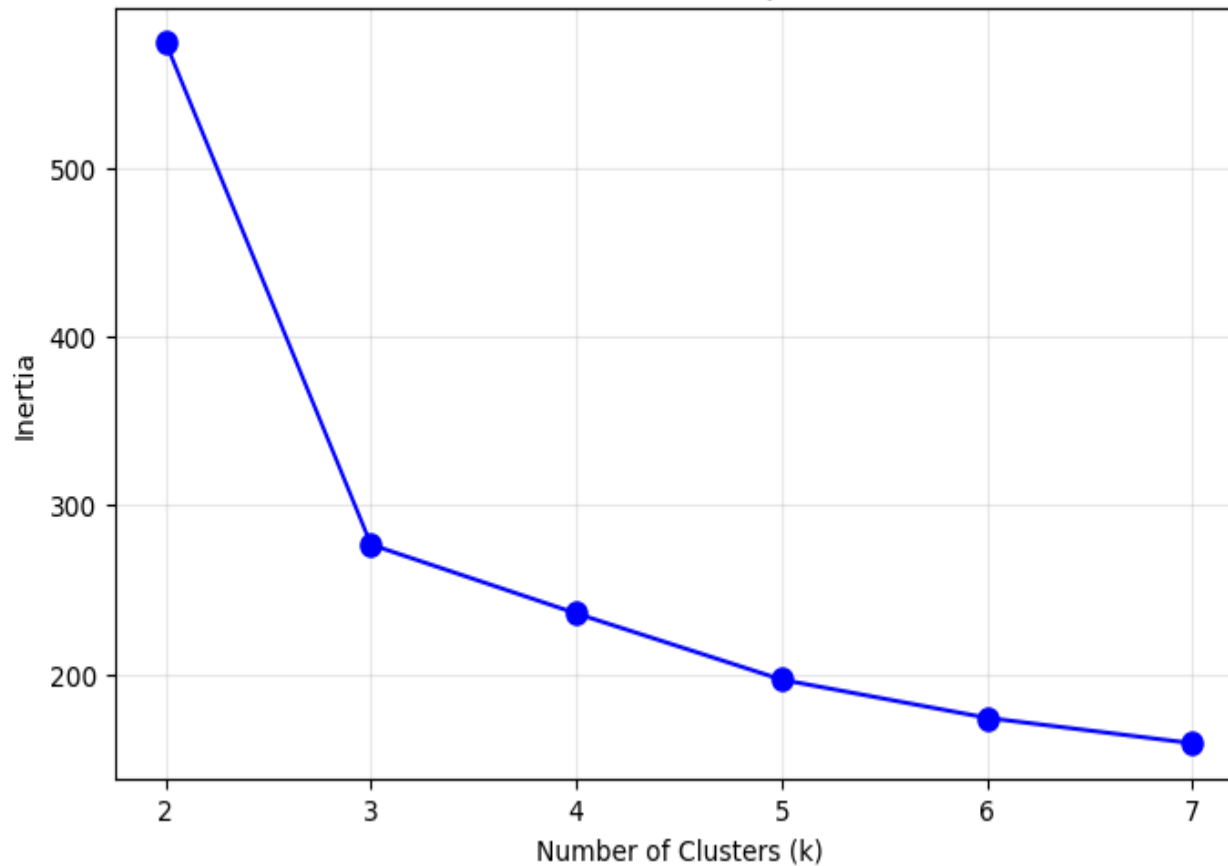


Figure 6. Elbow Method

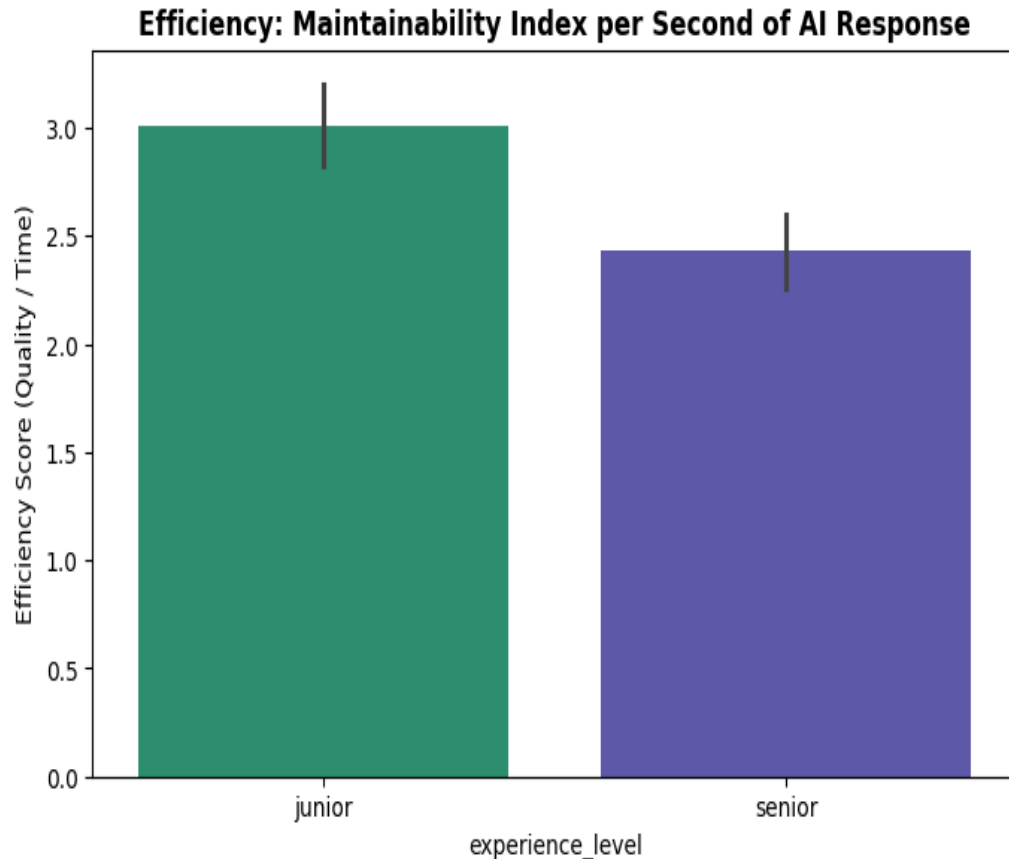
K	Inertia
2	574.67
3	276.69
4	235.83
5	196.51
6	173.87
7	158.77

Table 4. Cluster Validation Results

Silhouette Score (k=2)
=0.4781

Reasonable separation
between the clusters

Efficiency Metric: Maintainability Index per Second



Interpretation

Junior efficiency = 3.012

Senior = 2.431

Juniors appears 'more efficient' because wait time is shorter (13.23s vs 21.32s).

However – absolute MI matters more: 37.58 (junior) falls in the 'hard to maintain' zone (< 40).

Senior MI = 48.80 is the 'moderate-good' zone (40-65).

Figure 7. Maintainability Index per second of AI response

Limitations and Future Work

- **Single Task Type** – Only one React Todo Application. Greater divergence is expected for ambiguous or architecturally complex tasks.
- **Single Rater Scoring** – Prompt quality was scored by the author; no inter-rater reliability assessment was conducted.
- **AI Familiarity Not Controlled** – Participants prior experience with ChatGPT was not measured.
- **No Prompt Iteration** – Only single-shot prompts were analysed. Future studies should investigate multi-turn prompting behaviour.

Conclusion

1 Senior developers wrote significantly longer, clearer, and better-structured prompts than juniors, with very large effect sizes ($d > 2.8$).

2 **RQ1:** These better prompts produced better code – senior prompted code was more maintainable (MI: 48.80 vs 37.58), less complex, and had fewer errors, driven by prompt structure and specificity, not length.

3 **RQ2:** K-means clustering naturally separated juniors and seniors into two distinct groups, but 3 juniors behaved like seniors – proving that good prompting is a skill that can be learned, not just experienced.

4 **RQ3:** Structural organisation ($d = -3.741$) and specificity ($d = -3.211$) were among the strongest differentiators – structure matters most, and prompt engineering must be treated as a teachable skill, not assumed to come with experience.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. sc. ing., Associate Professor Samuel Moveh, for his guidance and support throughout this research.

I also thank the Transport and Telecommunication Institute (TSI) for providing the academic environment and resources for this study.

THANK YOU

ATHIRA ALLAVEEDU ASHOK

athiraathu1497@gmail.com

Reviewer Questions

1. In your controlled experiment, how did you control for potential variability in ChatGPT's responses (since the same prompt can yield different outputs across isolated sessions)?
2. How do you envision extending the proposed static analysis metrics to evaluate code safety and security vulnerabilities in AI-generated components?